```
RSpec.describe 'An ideal sandwich' do
  it 'is delicious' do
    sandwich = Sandwich.new('delicious', [])

    taste = sandwich.taste

    expect(taste).to eq('delicious')
  end

  it 'lets me add toppings' do
    sandwich = Sandwich.new('delicious', [])

    sandwich.toppings << 'cheese'
    toppings = sandwich.toppings

    expect(toppings).not_to be_empty
  end
end
```

Now, we'll look at a more traditional Ruby approach to reducing duplication.

Helper Methods

RSpec does a lot for us; it's easy to forget that it's just plain Ruby underneath. Each example group is a Ruby class, which means that we can define methods on it. Right after the `describe` line, add the following code:

01-getting-started/06/spec/sandwich_spec.rb

```
def sandwich
  Sandwich.new('delicious', [])
end
```

We've moved the common setup steps into a typical Ruby helper method, like you might do in one of your own classes. Now, you can remove the `sandwich = ...` lines from your examples.

This helper method isn't quite ready for prime time, though. Look at what happens when we rerun the specs:

```
$ rspec
.F

Failures:

  1) An ideal sandwich lets me add toppings
     Failure/Error: expect(toppings).not_to be_empty
       expected `[].empty?` to return false, got true
     # ./spec/sandwich_spec.rb:18:in `block (2 levels) in <top (required)>'

Finished in 0.0116 seconds (files took 0.08146 seconds to load)
2 examples, 1 failure

Failed examples:

rspec ./spec/sandwich_spec.rb:14 # An ideal sandwich lets me add toppings
```